Group8_ANN_Lab3

April 30, 2026

1 Group 8 - ANN Lab 3 (Pre-trained CNNs)

This notebook answers all required items in one file: 1. Hyperparameter tuning of `ImageDataGenerator`. Assigned: Abelardo, Aasen Sofia P. 2. VGG19 implementation and comparison with VGG16. Assigned: Gumban, Joevanni Paulo T. 3. VGG16 test accuracy vs training sample sizes `[500, 1000, 2000, 5000, 10000, 15000]` (no augmentation, 30 epochs/run). Assigned: Aquino, Matthew Benedict U. 4. Xception (Extreme Inception) implementation and comparison with VGG models. Assigned: Balingit, Den Mar F.

Each training item saves model artifacts as `.h5`.

[1]:
```
import os
import random
import shutil
import time
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras import layers, models, optimizers
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image as keras_image
from tensorflow.keras.applications import VGG16, VGG19, Xception

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

PROJECT_DIR = Path.cwd()
DATA_DIR = PROJECT_DIR / 'data'
OUTPUT_DIR = PROJECT_DIR / 'models'
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
```

```
BASE_DATASET_DIR = DATA_DIR / 'cats_and_dogs_from_petimages'
WORKING_SPLIT_DIR = DATA_DIR / 'working_split_for_lab3'

print('TensorFlow version:', tf.__version__)
print('Project directory:', PROJECT_DIR)
print('Base dataset:', BASE_DATASET_DIR)
print('Output directory:', OUTPUT_DIR)
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
I0000 00:00:1777516112.183840   21758 port.cc:153] oneDNN custom operations are
on. You may see slightly different numerical results due to floating-point
round-off errors from different computation orders. To turn them off, set the
environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1777516113.088504   21758 cpu_feature_guard.cc:227] This TensorFlow
binary is optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
I0000 00:00:1777516115.585330   21758 port.cc:153] oneDNN custom operations are
on. You may see slightly different numerical results due to floating-point
round-off errors from different computation orders. To turn them off, set the
environment variable `TF_ENABLE_ONEDNN_OPTS=0`.

TensorFlow version: 2.21.0
Project directory: /home/solyvie/workspace/cs-
projects/25263CSDMachineLearning/[7] Group 8 - Module 6 CNN Part 3 - Using Pre-
Trained Networks
Base dataset: /home/solyvie/workspace/cs-projects/25263CSDMachineLearning/[7]
Group 8 - Module 6 CNN Part 3 - Using Pre-Trained
Networks/data/cats_and_dogs_from_petimages
Output directory: /home/solyvie/workspace/cs-
projects/25263CSDMachineLearning/[7] Group 8 - Module 6 CNN Part 3 - Using Pre-
Trained Networks/models
```

[2]:
```
def count_split(split_dir: Path):
    counts = {}
    total = 0
    for cls in ['cats', 'dogs']:
        cls_dir = split_dir / cls
        n = len(list(cls_dir.glob('*'))) if cls_dir.exists() else 0
        counts[cls] = n
        total += n
    counts['total'] = total
    return counts
```

```python
def print_split_table(title, split_map):
    rows = []
    for split_name, split_dir in split_map.items():
        c = count_split(split_dir)
        rows.append({
            'split': split_name,
            'cats': c['cats'],
            'dogs': c['dogs'],
            'total': c['total'],
            'path': str(split_dir)
        })
    df = pd.DataFrame(rows)
    print(f'\n{title}')
    display(df)
    return df


# Prepare train/validation/test directories.
# If no dedicated test folder exists, create a disjoint val/test split from the␣
↪provided validation folder.
source_train = BASE_DATASET_DIR / 'train'
source_val = BASE_DATASET_DIR / 'validation'
source_test = BASE_DATASET_DIR / 'test'

if source_test.exists():
    train_dir = source_train
    validation_dir = source_val
    test_dir = source_test
    split_note = 'Using dataset-provided train/validation/test directories.'
else:
    if WORKING_SPLIT_DIR.exists():
        shutil.rmtree(WORKING_SPLIT_DIR)

    for split in ['train', 'validation', 'test']:
        for cls in ['cats', 'dogs']:
            (WORKING_SPLIT_DIR / split / cls).mkdir(parents=True, exist_ok=True)

    # Keep original train set intact (2000 total: 1000 cats + 1000 dogs).
    for cls in ['cats', 'dogs']:
        src_cls = source_train / cls
        dst_cls = WORKING_SPLIT_DIR / 'train' / cls
        for f in src_cls.iterdir():
            if f.is_file():
                shutil.copy2(f, dst_cls / f.name)
```

```
    # Split original validation set into validation/test (50/50) to create a␣
↪true holdout test set.
    # This avoids test leakage, but reduces val/test to 500 each (250/class).
    for cls in ['cats', 'dogs']:
        src_cls = source_val / cls
        files = [f for f in src_cls.iterdir() if f.is_file()]
        files.sort(key=lambda p: p.name)
        rng = random.Random(SEED)
        rng.shuffle(files)

        half = len(files) // 2
        val_files = files[:half]
        test_files = files[half:]

        for f in val_files:
            shutil.copy2(f, WORKING_SPLIT_DIR / 'validation' / cls / f.name)
        for f in test_files:
            shutil.copy2(f, WORKING_SPLIT_DIR / 'test' / cls / f.name)

    train_dir = WORKING_SPLIT_DIR / 'train'
    validation_dir = WORKING_SPLIT_DIR / 'validation'
    test_dir = WORKING_SPLIT_DIR / 'test'
    split_note = (
        'No dataset-provided test directory was found. Created disjoint␣
↪validation/test '
        'splits from the provided validation set (500 val + 500 test total).'
    )

print(split_note)
split_df = print_split_table(
    'Dataset split used by this notebook:',
    {'train': train_dir, 'validation': validation_dir, 'test': test_dir}
)

required = {'train': 2000, 'validation': 1000, 'test': 1000}
actual = {r['split']: int(r['total']) for _, r in split_df.iterrows()}
print('Required totals:', required)
print('Actual totals:  ', actual)
if actual != required:
    print(
        '\nNOTE: Current local dataset does not exactly match the required 2000/
↪1000/1000 split. '\
        'Results remain valid for this available split, and all comparisons use␣
↪the same split.'
    )
```

```
No dataset-provided test directory was found. Created disjoint validation/test
```

```
splits from the provided validation set (500 val + 500 test total).

Dataset split used by this notebook:

        split  cats  dogs  total  \
0       train  1000  1000   2000
1  validation   250   250    500
2        test   250   250    500

                                                  path
0  /home/solyvie/workspace/cs-projects/25263CSDMa…
1  /home/solyvie/workspace/cs-projects/25263CSDMa…
2  /home/solyvie/workspace/cs-projects/25263CSDMa…

Required totals: {'train': 2000, 'validation': 1000, 'test': 1000}
Actual totals:   {'train': 2000, 'validation': 500, 'test': 500}

NOTE: Current local dataset does not exactly match the required 2000/1000/1000
split. Results remain valid for this available split, and all comparisons use
the same split.
```

[3]:

```python
IMG_SIZE = (150, 150)
BATCH_SIZE = 20


def make_generators(train_aug_kwargs, batch_size=BATCH_SIZE):
    train_datagen = ImageDataGenerator(**train_aug_kwargs)
    eval_datagen = ImageDataGenerator(rescale=1.0 / 255)

    train_gen = train_datagen.flow_from_directory(
        str(train_dir),
        target_size=IMG_SIZE,
        batch_size=batch_size,
        class_mode='binary',
        shuffle=True,
        seed=SEED,
    )

    val_gen = eval_datagen.flow_from_directory(
        str(validation_dir),
        target_size=IMG_SIZE,
        batch_size=batch_size,
        class_mode='binary',
        shuffle=False,
    )

    test_gen = eval_datagen.flow_from_directory(
        str(test_dir),
```

```
        target_size=IMG_SIZE,
        batch_size=batch_size,
        class_mode='binary',
        shuffle=False,
    )

    return train_gen, val_gen, test_gen


def build_transfer_model(backbone_name='VGG16'):
    if backbone_name == 'VGG16':
        conv_base = VGG16(weights='imagenet', include_top=False,␣
 ↪input_shape=(150, 150, 3))
        pooling = 'flatten'
    elif backbone_name == 'VGG19':
        conv_base = VGG19(weights='imagenet', include_top=False,␣
 ↪input_shape=(150, 150, 3))
        pooling = 'flatten'
    elif backbone_name == 'Xception':
        conv_base = Xception(weights='imagenet', include_top=False,␣
 ↪input_shape=(150, 150, 3))
        pooling = 'gap'
    else:
        raise ValueError(f'Unsupported backbone: {backbone_name}')

    conv_base.trainable = False

    model = models.Sequential(name=f'{backbone_name}_transfer')
    model.add(conv_base)
    if pooling == 'flatten':
        model.add(layers.Flatten())
    else:
        model.add(layers.GlobalAveragePooling2D())
    model.add(layers.Dense(256, activation='relu'))
    model.add(layers.Dropout(0.5))
    model.add(layers.Dense(1, activation='sigmoid'))

    model.compile(
        optimizer=optimizers.RMSprop(learning_rate=2e-5),
        loss='binary_crossentropy',
        metrics=['accuracy'],
    )

    return model


def save_h5(model, model_name):
```

```
    model_path = OUTPUT_DIR / f'{model_name}.h5'
    model.save(str(model_path))
    return model_path


def train_eval_save(
    model_name,
    backbone_name,
    train_aug_kwargs,
    epochs=30,
    batch_size=BATCH_SIZE,
    early_stopping=True,
    steps_per_epoch=None,
):
    train_gen, val_gen, test_gen = make_generators(train_aug_kwargs,␣
 ↪batch_size=batch_size)

    model = build_transfer_model(backbone_name)

    callbacks = []
    if early_stopping:
        callbacks.append(
            EarlyStopping(
                monitor='val_loss',
                patience=5,
                restore_best_weights=True,
                verbose=1,
            )
        )

    start = time.time()
    history = model.fit(
        train_gen,
        epochs=epochs,
        validation_data=val_gen,
        callbacks=callbacks,
        steps_per_epoch=steps_per_epoch,
        verbose=1,
    )
    train_time = time.time() - start

    val_loss, val_acc = model.evaluate(val_gen, verbose=0)
    test_loss, test_acc = model.evaluate(test_gen, verbose=0)

    model_path = save_h5(model, model_name)

    result = {
```

```
        'model_name': model_name,
        'backbone': backbone_name,
        'epochs_ran': len(history.history['accuracy']),
        'best_val_acc_during_train': float(max(history.
↪history['val_accuracy'])),
        'final_train_acc': float(history.history['accuracy'][-1]),
        'final_val_acc': float(val_acc),
        'final_test_acc': float(test_acc),
        'train_time_sec': float(train_time),
        'model_h5_path': str(model_path),
        'history': history,
        'model_obj': model,
    }

    return result
```

1.1 1) Improve Model via `ImageDataGenerator` Hyperparameters

[4]:
```
default_aug = {
    'rescale': 1.0 / 255,
    'rotation_range': 20,
    'width_shift_range': 0.10,
    'height_shift_range': 0.10,
    'shear_range': 0.10,
    'zoom_range': 0.10,
    'horizontal_flip': True,
    'fill_mode': 'nearest',
}

modified_aug = {
    'rescale': 1.0 / 255,
    'rotation_range': 35,
    'width_shift_range': 0.20,
    'height_shift_range': 0.20,
    'shear_range': 0.15,
    'zoom_range': 0.20,
    'horizontal_flip': True,
    'brightness_range': (0.85, 1.15),
    'channel_shift_range': 20.0,
    'fill_mode': 'nearest',
}

q1_default = train_eval_save(
    model_name='Q1_VGG16_default_aug',
    backbone_name='VGG16',
    train_aug_kwargs=default_aug,
    epochs=30,
```

```
    early_stopping=True,
)

q1_modified = train_eval_save(
    model_name='Q1_VGG16_modified_aug',
    backbone_name='VGG16',
    train_aug_kwargs=modified_aug,
    epochs=30,
    early_stopping=True,
)

q1_df = pd.DataFrame([
    {k: v for k, v in q1_default.items() if k not in ['history', 'model_obj']},
    {k: v for k, v in q1_modified.items() if k not in ['history', 'model_obj']},
])

q1_display_cols = [
    'model_name', 'epochs_ran', 'best_val_acc_during_train',
    'final_train_acc', 'final_val_acc', 'final_test_acc',
    'train_time_sec', 'model_h5_path'
]

print('Q1 Results Table')
display(q1_df[q1_display_cols].sort_values('final_test_acc', ascending=False).
 ↪reset_index(drop=True))

q1_best = q1_df.sort_values('final_test_acc', ascending=False).iloc[0]
print(
    f"Best Q1 configuration by test accuracy: {q1_best['model_name']} "
    f"(test_acc={q1_best['final_test_acc']:.4f})"
)
```

```
Found 2000 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
```

```
I0000 00:00:1777516160.446885   21758 gpu_device.cc:2043] Created device
/job:localhost/replica:0/task:0/device:GPU:0 with 3536 MB memory:  -> device: 0,
name: NVIDIA GeForce RTX 4050 Laptop GPU, pci bus id: 0000:01:00.0, compute
capability: 8.9
```

```
Epoch 1/30
```

```
I0000 00:00:1777516162.605940   21758 generator_dataset_op.cc:213] Memory patch
applied: M_TRIM_THRESHOLD=128 kb was set.
I0000 00:00:1777516163.925649   23845 service.cc:153] XLA service 0x783d78035220
initialized for platform CUDA (this does not guarantee that XLA will be used).
Devices:
I0000 00:00:1777516163.925703   23845 service.cc:161]   StreamExecutor [0]:
```

```
NVIDIA GeForce RTX 4050 Laptop GPU, Compute Capability 8.9 (Driver: 13.2.0;
Runtime: 12.5.0; Toolkit: 12.5.0; DNN: 9.20.0)
I0000 00:00:1777516164.028972   23845 dump_mlir_util.cc:269] disabling MLIR
crash reproducer, set env var `MLIR_CRASH_REPRODUCER_DIRECTORY` to enable.
I0000 00:00:1777516164.531928   23845 cuda_dnn.cc:461] Loaded cuDNN version
92000
I0000 00:00:1777516164.614202   23845 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_1730__.62
I0000 00:00:1777516165.715872   23935 subprocess_compilation.cc:348] ptxas
warning : Registers are spilled to local memory in function
'gemm_fusion_MatMul_8', 12 bytes spill stores, 12 bytes spill loads


  1/100             17:31 11s/step - accuracy:
0.7500 - loss: 0.6643

I0000 00:00:1777516173.035783   23845 device_compiler.h:208] Compiled cluster
using XLA!  This line is logged at most once for the lifetime of the process.

100/100             26s 158ms/step -
accuracy: 0.6230 - loss: 0.6416 - val_accuracy: 0.8240 - val_loss: 0.4599
Epoch 2/30
100/100             14s 137ms/step -
accuracy: 0.7645 - loss: 0.4927 - val_accuracy: 0.8620 - val_loss: 0.3691
Epoch 3/30
100/100             13s 127ms/step -
accuracy: 0.8100 - loss: 0.4255 - val_accuracy: 0.8700 - val_loss: 0.3288
Epoch 4/30
100/100             14s 137ms/step -
accuracy: 0.8235 - loss: 0.3853 - val_accuracy: 0.8740 - val_loss: 0.3073
Epoch 5/30
100/100             13s 131ms/step -
accuracy: 0.8485 - loss: 0.3568 - val_accuracy: 0.8760 - val_loss: 0.2941
Epoch 6/30
100/100             14s 135ms/step -
accuracy: 0.8395 - loss: 0.3532 - val_accuracy: 0.8880 - val_loss: 0.2843
Epoch 7/30
100/100             12s 121ms/step -
accuracy: 0.8565 - loss: 0.3287 - val_accuracy: 0.8700 - val_loss: 0.2950
Epoch 8/30
100/100             14s 135ms/step -
accuracy: 0.8640 - loss: 0.3158 - val_accuracy: 0.8920 - val_loss: 0.2646
Epoch 9/30
100/100             13s 125ms/step -
accuracy: 0.8685 - loss: 0.3044 - val_accuracy: 0.8960 - val_loss: 0.2588
Epoch 10/30
100/100             13s 132ms/step -
accuracy: 0.8580 - loss: 0.3144 - val_accuracy: 0.8860 - val_loss: 0.2780
Epoch 11/30
```

```
100/100              13s 130ms/step -
accuracy: 0.8695 - loss: 0.2950 - val_accuracy: 0.8980 - val_loss: 0.2622
Epoch 12/30
100/100              13s 131ms/step -
accuracy: 0.8810 - loss: 0.2907 - val_accuracy: 0.8980 - val_loss: 0.2525
Epoch 13/30
100/100              13s 130ms/step -
accuracy: 0.8765 - loss: 0.2912 - val_accuracy: 0.8920 - val_loss: 0.2515
Epoch 14/30
100/100              12s 124ms/step -
accuracy: 0.8805 - loss: 0.2836 - val_accuracy: 0.8980 - val_loss: 0.2491
Epoch 15/30
100/100              12s 122ms/step -
accuracy: 0.8845 - loss: 0.2657 - val_accuracy: 0.8960 - val_loss: 0.2466
Epoch 16/30
100/100              13s 133ms/step -
accuracy: 0.8875 - loss: 0.2695 - val_accuracy: 0.9020 - val_loss: 0.2435
Epoch 17/30
100/100              12s 116ms/step -
accuracy: 0.8915 - loss: 0.2600 - val_accuracy: 0.8960 - val_loss: 0.2447
Epoch 18/30
100/100              13s 129ms/step -
accuracy: 0.8880 - loss: 0.2597 - val_accuracy: 0.8940 - val_loss: 0.2536
Epoch 19/30
100/100              12s 124ms/step -
accuracy: 0.8930 - loss: 0.2499 - val_accuracy: 0.9000 - val_loss: 0.2424
Epoch 20/30
100/100              13s 128ms/step -
accuracy: 0.9025 - loss: 0.2480 - val_accuracy: 0.8980 - val_loss: 0.2424
Epoch 21/30
100/100              13s 125ms/step -
accuracy: 0.8845 - loss: 0.2571 - val_accuracy: 0.9060 - val_loss: 0.2441
Epoch 22/30
100/100              13s 124ms/step -
accuracy: 0.8870 - loss: 0.2464 - val_accuracy: 0.9060 - val_loss: 0.2379
Epoch 23/30
100/100              13s 126ms/step -
accuracy: 0.9015 - loss: 0.2414 - val_accuracy: 0.9080 - val_loss: 0.2361
Epoch 24/30
100/100              11s 113ms/step -
accuracy: 0.8890 - loss: 0.2491 - val_accuracy: 0.8980 - val_loss: 0.2376
Epoch 25/30
100/100              12s 120ms/step -
accuracy: 0.9020 - loss: 0.2374 - val_accuracy: 0.8960 - val_loss: 0.2391
Epoch 26/30
100/100              14s 138ms/step -
accuracy: 0.8910 - loss: 0.2492 - val_accuracy: 0.8980 - val_loss: 0.2373
Epoch 27/30
```

```
100/100              14s 136ms/step -
accuracy: 0.8985 - loss: 0.2286 - val_accuracy: 0.9040 - val_loss: 0.2340
Epoch 28/30
100/100              13s 133ms/step -
accuracy: 0.8990 - loss: 0.2347 - val_accuracy: 0.8980 - val_loss: 0.2499
Epoch 29/30
100/100              13s 133ms/step -
accuracy: 0.9015 - loss: 0.2389 - val_accuracy: 0.8980 - val_loss: 0.2388
Epoch 30/30
100/100              13s 125ms/step -
accuracy: 0.9085 - loss: 0.2240 - val_accuracy: 0.8980 - val_loss: 0.2333
Restoring model weights from the end of the best epoch: 30.

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

Found 2000 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Epoch 1/30

I0000 00:00:1777516568.483024   23844 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_21220__.62

100/100              21s 171ms/step -
accuracy: 0.5845 - loss: 0.6929 - val_accuracy: 0.8120 - val_loss: 0.4914
Epoch 2/30
100/100              13s 130ms/step -
accuracy: 0.6915 - loss: 0.5764 - val_accuracy: 0.8460 - val_loss: 0.4164
Epoch 3/30
100/100              14s 142ms/step -
accuracy: 0.7315 - loss: 0.5201 - val_accuracy: 0.8520 - val_loss: 0.3677
Epoch 4/30
100/100              13s 134ms/step -
accuracy: 0.7680 - loss: 0.4722 - val_accuracy: 0.8500 - val_loss: 0.3335
Epoch 5/30
100/100              15s 145ms/step -
accuracy: 0.7945 - loss: 0.4430 - val_accuracy: 0.8600 - val_loss: 0.3262
Epoch 6/30
100/100              14s 135ms/step -
accuracy: 0.8030 - loss: 0.4314 - val_accuracy: 0.8540 - val_loss: 0.3159
Epoch 7/30
100/100              13s 134ms/step -
accuracy: 0.8075 - loss: 0.4251 - val_accuracy: 0.8720 - val_loss: 0.2897
Epoch 8/30
100/100              13s 133ms/step -
accuracy: 0.8150 - loss: 0.4116 - val_accuracy: 0.8720 - val_loss: 0.2843
```

```
Epoch 9/30
100/100              14s 136ms/step -
accuracy: 0.8270 - loss: 0.3965 - val_accuracy: 0.8820 - val_loss: 0.2764
Epoch 10/30
100/100              14s 141ms/step -
accuracy: 0.8230 - loss: 0.3912 - val_accuracy: 0.8780 - val_loss: 0.2742
Epoch 11/30
100/100              15s 154ms/step -
accuracy: 0.8275 - loss: 0.3750 - val_accuracy: 0.8980 - val_loss: 0.2666
Epoch 12/30
100/100              14s 140ms/step -
accuracy: 0.8310 - loss: 0.3730 - val_accuracy: 0.8760 - val_loss: 0.2632
Epoch 13/30
100/100              13s 131ms/step -
accuracy: 0.8195 - loss: 0.3931 - val_accuracy: 0.8780 - val_loss: 0.2645
Epoch 14/30
100/100              14s 143ms/step -
accuracy: 0.8500 - loss: 0.3505 - val_accuracy: 0.8860 - val_loss: 0.2587
Epoch 15/30
100/100              14s 139ms/step -
accuracy: 0.8340 - loss: 0.3639 - val_accuracy: 0.9040 - val_loss: 0.2635
Epoch 16/30
100/100              15s 151ms/step -
accuracy: 0.8405 - loss: 0.3551 - val_accuracy: 0.8820 - val_loss: 0.2553
Epoch 17/30
100/100              14s 142ms/step -
accuracy: 0.8465 - loss: 0.3414 - val_accuracy: 0.8900 - val_loss: 0.2559
Epoch 18/30
100/100              15s 148ms/step -
accuracy: 0.8305 - loss: 0.3663 - val_accuracy: 0.8880 - val_loss: 0.2565
Epoch 19/30
100/100              15s 145ms/step -
accuracy: 0.8535 - loss: 0.3329 - val_accuracy: 0.8960 - val_loss: 0.2569
Epoch 20/30
100/100              14s 143ms/step -
accuracy: 0.8490 - loss: 0.3406 - val_accuracy: 0.9000 - val_loss: 0.2483
Epoch 21/30
100/100              15s 149ms/step -
accuracy: 0.8495 - loss: 0.3454 - val_accuracy: 0.9000 - val_loss: 0.2546
Epoch 22/30
100/100              14s 135ms/step -
accuracy: 0.8485 - loss: 0.3437 - val_accuracy: 0.9000 - val_loss: 0.2460
Epoch 23/30
100/100              16s 159ms/step -
accuracy: 0.8545 - loss: 0.3238 - val_accuracy: 0.8980 - val_loss: 0.2459
Epoch 24/30
100/100              15s 149ms/step -
accuracy: 0.8620 - loss: 0.3362 - val_accuracy: 0.8940 - val_loss: 0.2496
```

```
Epoch 25/30
100/100              15s 151ms/step -
accuracy: 0.8645 - loss: 0.3152 - val_accuracy: 0.9060 - val_loss: 0.2408
Epoch 26/30
100/100              14s 142ms/step -
accuracy: 0.8540 - loss: 0.3265 - val_accuracy: 0.8940 - val_loss: 0.2426
Epoch 27/30
100/100              15s 146ms/step -
accuracy: 0.8500 - loss: 0.3258 - val_accuracy: 0.9000 - val_loss: 0.2391
Epoch 28/30
100/100              15s 146ms/step -
accuracy: 0.8565 - loss: 0.3246 - val_accuracy: 0.9020 - val_loss: 0.2419
Epoch 29/30
100/100              15s 154ms/step -
accuracy: 0.8520 - loss: 0.3220 - val_accuracy: 0.8900 - val_loss: 0.2441
Epoch 30/30
100/100              15s 148ms/step -
accuracy: 0.8655 - loss: 0.3116 - val_accuracy: 0.8980 - val_loss: 0.2383
Restoring model weights from the end of the best epoch: 30.

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

Q1 Results Table

              model_name  epochs_ran  best_val_acc_during_train  \
0   Q1_VGG16_default_aug          30                      0.908
1  Q1_VGG16_modified_aug          30                      0.906

   final_train_acc  final_val_acc  final_test_acc  train_time_sec  \
0           0.9085          0.898           0.900      400.872335
1           0.8655          0.898           0.884      437.656743

                                      model_h5_path
0  /home/solyvie/workspace/cs-projects/25263CSDMa…
1  /home/solyvie/workspace/cs-projects/25263CSDMa…

Best Q1 configuration by test accuracy: Q1_VGG16_default_aug (test_acc=0.9000)
```

1.2 2) Implement VGG19 and Compare with VGG16

Both are trained under the same augmentation setup for fair comparison.

[5]:
```
# Use the better augmentation from Q1 for Q2.
best_aug_name = 'modified_aug' if q1_modified['final_test_acc'] >=␣
↪q1_default['final_test_acc'] else 'default_aug'
best_aug = modified_aug if best_aug_name == 'modified_aug' else default_aug
```

```python
print('Using augmentation for Q2:', best_aug_name)

q2_vgg16 = train_eval_save(
    model_name='Q2_VGG16',
    backbone_name='VGG16',
    train_aug_kwargs=best_aug,
    epochs=30,
    early_stopping=True,
)

q2_vgg19 = train_eval_save(
    model_name='Q2_VGG19',
    backbone_name='VGG19',
    train_aug_kwargs=best_aug,
    epochs=30,
    early_stopping=True,
)

arch_stats = []
for backbone in ['VGG16', 'VGG19']:
    if backbone == 'VGG16':
        base = VGG16(weights='imagenet', include_top=False, input_shape=(150,␣
 ↪150, 3))
    else:
        base = VGG19(weights='imagenet', include_top=False, input_shape=(150,␣
 ↪150, 3))
    arch_stats.append({
        'backbone': backbone,
        'num_layers': len(base.layers),
        'num_params': base.count_params(),
    })

arch_df = pd.DataFrame(arch_stats)
q2_df = pd.DataFrame([
    {k: v for k, v in q2_vgg16.items() if k not in ['history', 'model_obj']},
    {k: v for k, v in q2_vgg19.items() if k not in ['history', 'model_obj']},
])
q2_df = q2_df.merge(arch_df, left_on='backbone', right_on='backbone',␣
 ↪how='left')

q2_display_cols = [
    'model_name', 'backbone', 'num_layers', 'num_params',
    'epochs_ran', 'final_train_acc', 'final_val_acc', 'final_test_acc',
    'train_time_sec', 'model_h5_path'
]

print('Q2 Results Table')
```

```
display(q2_df[q2_display_cols].sort_values('final_test_acc', ascending=False).
 ↪reset_index(drop=True))

q2_best = q2_df.sort_values('final_test_acc', ascending=False).iloc[0]
print(
    f"Best in Q2 by test accuracy: {q2_best['backbone']} "
    f"(test_acc={q2_best['final_test_acc']:.4f})"
)
```

```
Using augmentation for Q2: default_aug
Found 2000 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Epoch 1/30
```

```
I0000 00:00:1777517083.245695   23845 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_40830__.62
```

```
100/100              19s 153ms/step -
accuracy: 0.6510 - loss: 0.6270 - val_accuracy: 0.8300 - val_loss: 0.4425
Epoch 2/30
100/100              13s 125ms/step -
accuracy: 0.7550 - loss: 0.4939 - val_accuracy: 0.8580 - val_loss: 0.3666
Epoch 3/30
100/100              14s 137ms/step -
accuracy: 0.8020 - loss: 0.4260 - val_accuracy: 0.8700 - val_loss: 0.3267
Epoch 4/30
100/100              14s 138ms/step -
accuracy: 0.8215 - loss: 0.3923 - val_accuracy: 0.8760 - val_loss: 0.3066
Epoch 5/30
100/100              12s 124ms/step -
accuracy: 0.8375 - loss: 0.3743 - val_accuracy: 0.8820 - val_loss: 0.2937
Epoch 6/30
100/100              11s 109ms/step -
accuracy: 0.8405 - loss: 0.3525 - val_accuracy: 0.8980 - val_loss: 0.2779
Epoch 7/30
100/100              13s 130ms/step -
accuracy: 0.8460 - loss: 0.3479 - val_accuracy: 0.8900 - val_loss: 0.2718
Epoch 8/30
100/100              13s 128ms/step -
accuracy: 0.8710 - loss: 0.3251 - val_accuracy: 0.8880 - val_loss: 0.2720
Epoch 9/30
100/100              13s 126ms/step -
accuracy: 0.8700 - loss: 0.3035 - val_accuracy: 0.8940 - val_loss: 0.2645
Epoch 10/30
100/100              13s 132ms/step -
accuracy: 0.8635 - loss: 0.3081 - val_accuracy: 0.8980 - val_loss: 0.2577
Epoch 11/30
100/100              13s 133ms/step -
```

```
accuracy: 0.8665 - loss: 0.3070 - val_accuracy: 0.8980 - val_loss: 0.2560
Epoch 12/30
100/100             14s 134ms/step -
accuracy: 0.8735 - loss: 0.2961 - val_accuracy: 0.8980 - val_loss: 0.2546
Epoch 13/30
100/100             13s 130ms/step -
accuracy: 0.8780 - loss: 0.2868 - val_accuracy: 0.9020 - val_loss: 0.2497
Epoch 14/30
100/100             14s 136ms/step -
accuracy: 0.8810 - loss: 0.2833 - val_accuracy: 0.9100 - val_loss: 0.2509
Epoch 15/30
100/100             13s 128ms/step -
accuracy: 0.8880 - loss: 0.2756 - val_accuracy: 0.9100 - val_loss: 0.2475
Epoch 16/30
100/100             13s 127ms/step -
accuracy: 0.8815 - loss: 0.2717 - val_accuracy: 0.9120 - val_loss: 0.2449
Epoch 17/30
100/100             12s 123ms/step -
accuracy: 0.8855 - loss: 0.2655 - val_accuracy: 0.9120 - val_loss: 0.2429
Epoch 18/30
100/100             13s 132ms/step -
accuracy: 0.8905 - loss: 0.2636 - val_accuracy: 0.9080 - val_loss: 0.2413
Epoch 19/30
100/100             13s 127ms/step -
accuracy: 0.8955 - loss: 0.2548 - val_accuracy: 0.9080 - val_loss: 0.2405
Epoch 20/30
100/100             13s 129ms/step -
accuracy: 0.8885 - loss: 0.2583 - val_accuracy: 0.9060 - val_loss: 0.2368
Epoch 21/30
100/100             13s 131ms/step -
accuracy: 0.8985 - loss: 0.2432 - val_accuracy: 0.9140 - val_loss: 0.2384
Epoch 22/30
100/100             13s 126ms/step -
accuracy: 0.8890 - loss: 0.2607 - val_accuracy: 0.9040 - val_loss: 0.2355
Epoch 23/30
100/100             13s 125ms/step -
accuracy: 0.9020 - loss: 0.2327 - val_accuracy: 0.9080 - val_loss: 0.2495
Epoch 24/30
100/100             13s 127ms/step -
accuracy: 0.9010 - loss: 0.2366 - val_accuracy: 0.9140 - val_loss: 0.2339
Epoch 25/30
100/100             13s 130ms/step -
accuracy: 0.9020 - loss: 0.2399 - val_accuracy: 0.9120 - val_loss: 0.2331
Epoch 26/30
100/100             12s 124ms/step -
accuracy: 0.8980 - loss: 0.2399 - val_accuracy: 0.9140 - val_loss: 0.2354
Epoch 27/30
100/100             13s 128ms/step -
```

```
accuracy: 0.8935 - loss: 0.2441 - val_accuracy: 0.8960 - val_loss: 0.2537
Epoch 28/30
100/100             12s 124ms/step -
accuracy: 0.9040 - loss: 0.2319 - val_accuracy: 0.9060 - val_loss: 0.2320
Epoch 29/30
100/100             13s 126ms/step -
accuracy: 0.9000 - loss: 0.2366 - val_accuracy: 0.9140 - val_loss: 0.2352
Epoch 30/30
100/100             13s 125ms/step -
accuracy: 0.9030 - loss: 0.2374 - val_accuracy: 0.9100 - val_loss: 0.2305
Restoring model weights from the end of the best epoch: 30.

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

Found 2000 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Epoch 1/30

I0000 00:00:1777517479.180487   23844 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_60812__.65

100/100             19s 146ms/step -
accuracy: 0.6145 - loss: 0.6559 - val_accuracy: 0.7920 - val_loss: 0.4996
Epoch 2/30
100/100             12s 120ms/step -
accuracy: 0.7245 - loss: 0.5387 - val_accuracy: 0.8100 - val_loss: 0.4371
Epoch 3/30
100/100             13s 130ms/step -
accuracy: 0.7650 - loss: 0.4838 - val_accuracy: 0.8360 - val_loss: 0.3940
Epoch 4/30
100/100             13s 130ms/step -
accuracy: 0.7935 - loss: 0.4447 - val_accuracy: 0.8360 - val_loss: 0.3723
Epoch 5/30
100/100             13s 127ms/step -
accuracy: 0.8120 - loss: 0.4096 - val_accuracy: 0.8440 - val_loss: 0.3551
Epoch 6/30
100/100             13s 124ms/step -
accuracy: 0.8050 - loss: 0.4066 - val_accuracy: 0.8680 - val_loss: 0.3395
Epoch 7/30
100/100             13s 127ms/step -
accuracy: 0.8280 - loss: 0.3829 - val_accuracy: 0.8620 - val_loss: 0.3341
Epoch 8/30
100/100             13s 133ms/step -
accuracy: 0.8290 - loss: 0.3824 - val_accuracy: 0.8640 - val_loss: 0.3263
Epoch 9/30
```

```
100/100              13s 128ms/step -
accuracy: 0.8295 - loss: 0.3683 - val_accuracy: 0.8640 - val_loss: 0.3216
Epoch 10/30
100/100              13s 129ms/step -
accuracy: 0.8370 - loss: 0.3669 - val_accuracy: 0.8560 - val_loss: 0.3318
Epoch 11/30
100/100              14s 135ms/step -
accuracy: 0.8510 - loss: 0.3464 - val_accuracy: 0.8700 - val_loss: 0.3205
Epoch 12/30
100/100              13s 132ms/step -
accuracy: 0.8525 - loss: 0.3346 - val_accuracy: 0.8600 - val_loss: 0.3270
Epoch 13/30
100/100              13s 131ms/step -
accuracy: 0.8545 - loss: 0.3409 - val_accuracy: 0.8480 - val_loss: 0.3363
Epoch 14/30
100/100              13s 127ms/step -
accuracy: 0.8580 - loss: 0.3262 - val_accuracy: 0.8680 - val_loss: 0.3194
Epoch 15/30
100/100              13s 128ms/step -
accuracy: 0.8585 - loss: 0.3266 - val_accuracy: 0.8720 - val_loss: 0.3140
Epoch 16/30
100/100              13s 125ms/step -
accuracy: 0.8525 - loss: 0.3181 - val_accuracy: 0.8760 - val_loss: 0.3066
Epoch 17/30
100/100              13s 125ms/step -
accuracy: 0.8630 - loss: 0.3047 - val_accuracy: 0.8800 - val_loss: 0.3054
Epoch 18/30
100/100              12s 116ms/step -
accuracy: 0.8755 - loss: 0.3015 - val_accuracy: 0.8540 - val_loss: 0.3272
Epoch 19/30
100/100              13s 130ms/step -
accuracy: 0.8760 - loss: 0.2973 - val_accuracy: 0.8740 - val_loss: 0.3083
Epoch 20/30
100/100              13s 124ms/step -
accuracy: 0.8640 - loss: 0.3002 - val_accuracy: 0.8660 - val_loss: 0.3001
Epoch 21/30
100/100              13s 129ms/step -
accuracy: 0.8785 - loss: 0.2913 - val_accuracy: 0.8620 - val_loss: 0.3154
Epoch 22/30
100/100              13s 133ms/step -
accuracy: 0.8755 - loss: 0.2951 - val_accuracy: 0.8680 - val_loss: 0.3004
Epoch 23/30
100/100              13s 126ms/step -
accuracy: 0.8785 - loss: 0.2855 - val_accuracy: 0.8720 - val_loss: 0.3055
Epoch 24/30
100/100              13s 132ms/step -
accuracy: 0.8725 - loss: 0.2962 - val_accuracy: 0.8720 - val_loss: 0.2984
Epoch 25/30
```

```
100/100              13s 130ms/step -
accuracy: 0.8810 - loss: 0.2842 - val_accuracy: 0.8800 - val_loss: 0.2998
Epoch 26/30
100/100              13s 129ms/step -
accuracy: 0.8795 - loss: 0.2774 - val_accuracy: 0.8720 - val_loss: 0.2964
Epoch 27/30
100/100              13s 125ms/step -
accuracy: 0.8795 - loss: 0.2707 - val_accuracy: 0.8700 - val_loss: 0.3029
Epoch 28/30
100/100              13s 129ms/step -
accuracy: 0.8875 - loss: 0.2723 - val_accuracy: 0.8760 - val_loss: 0.2975
Epoch 29/30
100/100              13s 129ms/step -
accuracy: 0.8740 - loss: 0.2867 - val_accuracy: 0.8720 - val_loss: 0.2964
Epoch 30/30
100/100              13s 131ms/step -
accuracy: 0.8870 - loss: 0.2736 - val_accuracy: 0.8440 - val_loss: 0.3351
Restoring model weights from the end of the best epoch: 26.
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
```

```
Q2 Results Table

  model_name backbone  num_layers  num_params  epochs_ran  final_train_acc  \
0   Q2_VGG19    VGG19          22    20024384          30            0.887
1   Q2_VGG16    VGG16          19    14714688          30            0.903

   final_val_acc  final_test_acc  train_time_sec  \
0          0.872           0.910      392.506488
1          0.910           0.898      393.185707

                                       model_h5_path
0  /home/solyvie/workspace/cs-projects/25263CSDMa…
1  /home/solyvie/workspace/cs-projects/25263CSDMa…

Best in Q2 by test accuracy: VGG19 (test_acc=0.9100)
```

1.3 3) VGG16 Test Accuracy vs Training Samples (No Data Augmentation)

Required sample sizes: `500, 1000, 2000, 5000, 10000, 15000`. Each run uses **30 epochs**.

[6]:
```
sample_sizes = [500, 1000, 2000, 5000, 10000, 15000]
q3_results = []

no_aug = {'rescale': 1.0 / 255}
```

```python
for n_samples in sample_sizes:
    # Approximate sample budget per epoch by controlling steps_per_epoch.
    steps = max(1, int(np.ceil(n_samples / BATCH_SIZE)))
    run_name = f'Q3_VGG16_noaug_{n_samples}samples'

    print(f'\nTraining {run_name} | steps_per_epoch={steps}, epochs=30')
    run = train_eval_save(
        model_name=run_name,
        backbone_name='VGG16',
        train_aug_kwargs=no_aug,
        epochs=30,
        early_stopping=False,  # fixed 30 epochs per requirement
        steps_per_epoch=steps,
    )

    q3_results.append({
        'samples_per_epoch_target': n_samples,
        'steps_per_epoch': steps,
        'epochs_ran': run['epochs_ran'],
        'final_train_acc': run['final_train_acc'],
        'final_val_acc': run['final_val_acc'],
        'final_test_acc': run['final_test_acc'],
        'train_time_sec': run['train_time_sec'],
        'model_h5_path': run['model_h5_path'],
    })

q3_df = pd.DataFrame(q3_results)
print('Q3 Results Table')
display(q3_df)

plt.figure(figsize=(9, 5))
plt.plot(q3_df['samples_per_epoch_target'], q3_df['final_test_acc'],␣
 ↪marker='o', label='Test Accuracy')
plt.plot(q3_df['samples_per_epoch_target'], q3_df['final_val_acc'], marker='o',␣
 ↪label='Validation Accuracy')
plt.xlabel('Training samples per epoch (target)')
plt.ylabel('Accuracy')
plt.title('VGG16 Accuracy vs Training Sample Budget (No Augmentation, 30␣
 ↪Epochs)')
plt.grid(True)
plt.legend()
plt.show()

best_q3 = q3_df.sort_values('final_test_acc', ascending=False).iloc[0]
print(
    f"Best Q3 run: {int(best_q3['samples_per_epoch_target'])} samples/epoch "
    f"(test_acc={best_q3['final_test_acc']:.4f})"
```

```
)
```

```
Training Q3_VGG16_noaug_500samples | steps_per_epoch=25, epochs=30
Found 2000 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Epoch 1/30
```

```
I0000 00:00:1777518023.531267   23843 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_81226__.62
```

```
25/25              8s 152ms/step -
accuracy: 0.5580 - loss: 0.7073 - val_accuracy: 0.7300 - val_loss: 0.6087
Epoch 2/30
25/25              2s 76ms/step -
accuracy: 0.6260 - loss: 0.6404 - val_accuracy: 0.8140 - val_loss: 0.5416
Epoch 3/30
25/25              2s 77ms/step -
accuracy: 0.7140 - loss: 0.5569 - val_accuracy: 0.8260 - val_loss: 0.4972
Epoch 4/30
25/25              2s 75ms/step -
accuracy: 0.7680 - loss: 0.5091 - val_accuracy: 0.7860 - val_loss: 0.4798
Epoch 5/30
25/25              2s 76ms/step -
accuracy: 0.7960 - loss: 0.4707 - val_accuracy: 0.8500 - val_loss: 0.4360
Epoch 6/30
25/25              2s 80ms/step -
accuracy: 0.8120 - loss: 0.4388 - val_accuracy: 0.8660 - val_loss: 0.4127
Epoch 7/30
25/25              2s 78ms/step -
accuracy: 0.8060 - loss: 0.4303 - val_accuracy: 0.8320 - val_loss: 0.3996
Epoch 8/30
25/25              2s 76ms/step -
accuracy: 0.8280 - loss: 0.4093 - val_accuracy: 0.8440 - val_loss: 0.3920
Epoch 9/30
25/25              2s 86ms/step -
accuracy: 0.8260 - loss: 0.3924 - val_accuracy: 0.8420 - val_loss: 0.3735
Epoch 10/30
25/25              2s 76ms/step -
accuracy: 0.8380 - loss: 0.3889 - val_accuracy: 0.8840 - val_loss: 0.3579
Epoch 11/30
25/25              2s 78ms/step -
accuracy: 0.8260 - loss: 0.3841 - val_accuracy: 0.8720 - val_loss: 0.3462
Epoch 12/30
25/25              2s 79ms/step -
accuracy: 0.8140 - loss: 0.3653 - val_accuracy: 0.8720 - val_loss: 0.3389
Epoch 13/30
25/25              2s 78ms/step -
```

```
accuracy: 0.8480 - loss: 0.3512 - val_accuracy: 0.8880 - val_loss: 0.3316
Epoch 14/30
25/25              2s 78ms/step -
accuracy: 0.8520 - loss: 0.3330 - val_accuracy: 0.8720 - val_loss: 0.3242
Epoch 15/30
25/25              2s 79ms/step -
accuracy: 0.8820 - loss: 0.3039 - val_accuracy: 0.8860 - val_loss: 0.3187
Epoch 16/30
25/25              2s 78ms/step -
accuracy: 0.8720 - loss: 0.2986 - val_accuracy: 0.8640 - val_loss: 0.3154
Epoch 17/30
25/25              2s 78ms/step -
accuracy: 0.9000 - loss: 0.2894 - val_accuracy: 0.8800 - val_loss: 0.3075
Epoch 18/30
25/25              2s 90ms/step -
accuracy: 0.8980 - loss: 0.2587 - val_accuracy: 0.8880 - val_loss: 0.3029
Epoch 19/30
25/25              2s 79ms/step -
accuracy: 0.8760 - loss: 0.2978 - val_accuracy: 0.8940 - val_loss: 0.2994
Epoch 20/30
25/25              2s 77ms/step -
accuracy: 0.9200 - loss: 0.2410 - val_accuracy: 0.8880 - val_loss: 0.2939
Epoch 21/30
25/25              2s 79ms/step -
accuracy: 0.8780 - loss: 0.2804 - val_accuracy: 0.8900 - val_loss: 0.2920
Epoch 22/30
25/25              2s 78ms/step -
accuracy: 0.9140 - loss: 0.2485 - val_accuracy: 0.8860 - val_loss: 0.2983
Epoch 23/30
25/25              2s 80ms/step -
accuracy: 0.9060 - loss: 0.2641 - val_accuracy: 0.8880 - val_loss: 0.2894
Epoch 24/30
25/25              2s 80ms/step -
accuracy: 0.8860 - loss: 0.2611 - val_accuracy: 0.8900 - val_loss: 0.2841
Epoch 25/30
25/25              2s 81ms/step -
accuracy: 0.8980 - loss: 0.2621 - val_accuracy: 0.8740 - val_loss: 0.2907
Epoch 26/30
25/25              2s 84ms/step -
accuracy: 0.9040 - loss: 0.2471 - val_accuracy: 0.8860 - val_loss: 0.2808
Epoch 27/30
25/25              2s 78ms/step -
accuracy: 0.9020 - loss: 0.2560 - val_accuracy: 0.8940 - val_loss: 0.2788
Epoch 28/30
25/25              2s 77ms/step -
accuracy: 0.9040 - loss: 0.2482 - val_accuracy: 0.8920 - val_loss: 0.2787
Epoch 29/30
25/25              2s 80ms/step -
```

```
accuracy: 0.8940 - loss: 0.2345 - val_accuracy: 0.8880 - val_loss: 0.2889
Epoch 30/30
25/25              2s 76ms/step -
accuracy: 0.9060 - loss: 0.2277 - val_accuracy: 0.8820 - val_loss: 0.2789

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.


Training Q3_VGG16_noaug_1000samples | steps_per_epoch=50, epochs=30
Found 2000 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Epoch 1/30

I0000 00:00:1777518091.121248   23845 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_90407__.62

50/50              9s 104ms/step -
accuracy: 0.6400 - loss: 0.6542 - val_accuracy: 0.8240 - val_loss: 0.4816
Epoch 2/30
50/50              3s 58ms/step -
accuracy: 0.7540 - loss: 0.4984 - val_accuracy: 0.8680 - val_loss: 0.4032
Epoch 3/30
50/50              3s 64ms/step -
accuracy: 0.8010 - loss: 0.4247 - val_accuracy: 0.8560 - val_loss: 0.3791
Epoch 4/30
50/50              3s 57ms/step -
accuracy: 0.8240 - loss: 0.3949 - val_accuracy: 0.8780 - val_loss: 0.3373
Epoch 5/30
50/50              3s 57ms/step -
accuracy: 0.8440 - loss: 0.3747 - val_accuracy: 0.8820 - val_loss: 0.3221
Epoch 6/30
50/50              3s 59ms/step -
accuracy: 0.8560 - loss: 0.3368 - val_accuracy: 0.8880 - val_loss: 0.3069
Epoch 7/30
50/50              3s 63ms/step -
accuracy: 0.8700 - loss: 0.3085 - val_accuracy: 0.8880 - val_loss: 0.2949
Epoch 8/30
50/50              3s 60ms/step -
accuracy: 0.8830 - loss: 0.2910 - val_accuracy: 0.8940 - val_loss: 0.2862
Epoch 9/30
50/50              3s 57ms/step -
accuracy: 0.8970 - loss: 0.2784 - val_accuracy: 0.8920 - val_loss: 0.2809
Epoch 10/30
50/50              3s 58ms/step -
accuracy: 0.9010 - loss: 0.2640 - val_accuracy: 0.8880 - val_loss: 0.2762
```

```
Epoch 11/30
50/50              3s 64ms/step -
accuracy: 0.8890 - loss: 0.2648 - val_accuracy: 0.8880 - val_loss: 0.2743
Epoch 12/30
50/50              3s 65ms/step -
accuracy: 0.8890 - loss: 0.2697 - val_accuracy: 0.8860 - val_loss: 0.2722
Epoch 13/30
50/50              3s 60ms/step -
accuracy: 0.9070 - loss: 0.2378 - val_accuracy: 0.9020 - val_loss: 0.2677
Epoch 14/30
50/50              3s 59ms/step -
accuracy: 0.9100 - loss: 0.2283 - val_accuracy: 0.8920 - val_loss: 0.2633
Epoch 15/30
50/50              3s 59ms/step -
accuracy: 0.9240 - loss: 0.2230 - val_accuracy: 0.8920 - val_loss: 0.2594
Epoch 16/30
50/50              3s 57ms/step -
accuracy: 0.9050 - loss: 0.2344 - val_accuracy: 0.8940 - val_loss: 0.2597
Epoch 17/30
50/50              3s 57ms/step -
accuracy: 0.9210 - loss: 0.2100 - val_accuracy: 0.8980 - val_loss: 0.2572
Epoch 18/30
50/50              3s 61ms/step -
accuracy: 0.9200 - loss: 0.2030 - val_accuracy: 0.8740 - val_loss: 0.2852
Epoch 19/30
50/50              3s 56ms/step -
accuracy: 0.9280 - loss: 0.1938 - val_accuracy: 0.9000 - val_loss: 0.2536
Epoch 20/30
50/50              3s 58ms/step -
accuracy: 0.9260 - loss: 0.1978 - val_accuracy: 0.8960 - val_loss: 0.2507
Epoch 21/30
50/50              3s 58ms/step -
accuracy: 0.9230 - loss: 0.1948 - val_accuracy: 0.9020 - val_loss: 0.2502
Epoch 22/30
50/50              3s 58ms/step -
accuracy: 0.9210 - loss: 0.2008 - val_accuracy: 0.9020 - val_loss: 0.2515
Epoch 23/30
50/50              3s 57ms/step -
accuracy: 0.9250 - loss: 0.1918 - val_accuracy: 0.8980 - val_loss: 0.2531
Epoch 24/30
50/50              3s 56ms/step -
accuracy: 0.9350 - loss: 0.1782 - val_accuracy: 0.9060 - val_loss: 0.2511
Epoch 25/30
50/50              3s 55ms/step -
accuracy: 0.9400 - loss: 0.1741 - val_accuracy: 0.9020 - val_loss: 0.2488
Epoch 26/30
50/50              3s 56ms/step -
accuracy: 0.9420 - loss: 0.1629 - val_accuracy: 0.9060 - val_loss: 0.2490
```

```
Epoch 27/30
50/50              3s 55ms/step -
accuracy: 0.9330 - loss: 0.1694 - val_accuracy: 0.9040 - val_loss: 0.2496
Epoch 28/30
50/50              3s 55ms/step -
accuracy: 0.9530 - loss: 0.1548 - val_accuracy: 0.9020 - val_loss: 0.2495
Epoch 29/30
50/50              3s 56ms/step -
accuracy: 0.9470 - loss: 0.1585 - val_accuracy: 0.9040 - val_loss: 0.2495
Epoch 30/30
50/50              3s 56ms/step -
accuracy: 0.9550 - loss: 0.1439 - val_accuracy: 0.9100 - val_loss: 0.2471

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.


Training Q3_VGG16_noaug_2000samples | steps_per_epoch=100, epochs=30
Found 2000 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Epoch 1/30

I0000 00:00:1777518190.243516   23844 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_102596__.62

100/100             11s 68ms/step -
accuracy: 0.7000 - loss: 0.5665 - val_accuracy: 0.8520 - val_loss: 0.4156
Epoch 2/30
100/100             5s 48ms/step -
accuracy: 0.8160 - loss: 0.4056 - val_accuracy: 0.8660 - val_loss: 0.3483
Epoch 3/30
100/100             5s 46ms/step -
accuracy: 0.8620 - loss: 0.3339 - val_accuracy: 0.8680 - val_loss: 0.3190
Epoch 4/30
100/100             5s 47ms/step -
accuracy: 0.8660 - loss: 0.3079 - val_accuracy: 0.8820 - val_loss: 0.2944
Epoch 5/30
100/100             4s 44ms/step -
accuracy: 0.8810 - loss: 0.2793 - val_accuracy: 0.8820 - val_loss: 0.2828
Epoch 6/30
100/100             5s 46ms/step -
accuracy: 0.8980 - loss: 0.2561 - val_accuracy: 0.8980 - val_loss: 0.2758
Epoch 7/30
100/100             5s 45ms/step -
accuracy: 0.8965 - loss: 0.2457 - val_accuracy: 0.8980 - val_loss: 0.2708
Epoch 8/30
```

```
100/100              5s 45ms/step -
accuracy: 0.9140 - loss: 0.2233 - val_accuracy: 0.8840 - val_loss: 0.2688
Epoch 9/30
100/100              5s 47ms/step -
accuracy: 0.9120 - loss: 0.2125 - val_accuracy: 0.8920 - val_loss: 0.2627
Epoch 10/30
100/100              5s 46ms/step -
accuracy: 0.9220 - loss: 0.2072 - val_accuracy: 0.8980 - val_loss: 0.2615
Epoch 11/30
100/100              5s 46ms/step -
accuracy: 0.9255 - loss: 0.1890 - val_accuracy: 0.8940 - val_loss: 0.2595
Epoch 12/30
100/100              5s 47ms/step -
accuracy: 0.9315 - loss: 0.1873 - val_accuracy: 0.8980 - val_loss: 0.2577
Epoch 13/30
100/100              5s 45ms/step -
accuracy: 0.9370 - loss: 0.1757 - val_accuracy: 0.8980 - val_loss: 0.2568
Epoch 14/30
100/100              5s 46ms/step -
accuracy: 0.9375 - loss: 0.1662 - val_accuracy: 0.9000 - val_loss: 0.2598
Epoch 15/30
100/100              5s 46ms/step -
accuracy: 0.9405 - loss: 0.1628 - val_accuracy: 0.8960 - val_loss: 0.2574
Epoch 16/30
100/100              5s 46ms/step -
accuracy: 0.9470 - loss: 0.1533 - val_accuracy: 0.9020 - val_loss: 0.2580
Epoch 17/30
100/100              5s 46ms/step -
accuracy: 0.9500 - loss: 0.1488 - val_accuracy: 0.9000 - val_loss: 0.2604
Epoch 18/30
100/100              5s 46ms/step -
accuracy: 0.9440 - loss: 0.1440 - val_accuracy: 0.9040 - val_loss: 0.2589
Epoch 19/30
100/100              5s 46ms/step -
accuracy: 0.9550 - loss: 0.1382 - val_accuracy: 0.9040 - val_loss: 0.2613
Epoch 20/30
100/100              5s 46ms/step -
accuracy: 0.9585 - loss: 0.1292 - val_accuracy: 0.8940 - val_loss: 0.2577
Epoch 21/30
100/100              5s 46ms/step -
accuracy: 0.9620 - loss: 0.1276 - val_accuracy: 0.8940 - val_loss: 0.2615
Epoch 22/30
100/100              5s 47ms/step -
accuracy: 0.9600 - loss: 0.1236 - val_accuracy: 0.8980 - val_loss: 0.2647
Epoch 23/30
100/100              5s 46ms/step -
accuracy: 0.9650 - loss: 0.1170 - val_accuracy: 0.8940 - val_loss: 0.2634
Epoch 24/30
```

```
100/100              5s 48ms/step -
accuracy: 0.9655 - loss: 0.1118 - val_accuracy: 0.8920 - val_loss: 0.2632
Epoch 25/30
100/100              5s 46ms/step -
accuracy: 0.9630 - loss: 0.1110 - val_accuracy: 0.9000 - val_loss: 0.2674
Epoch 26/30
100/100              5s 46ms/step -
accuracy: 0.9710 - loss: 0.1015 - val_accuracy: 0.8900 - val_loss: 0.2656
Epoch 27/30
100/100              5s 46ms/step -
accuracy: 0.9730 - loss: 0.0945 - val_accuracy: 0.9000 - val_loss: 0.2647
Epoch 28/30
100/100              5s 46ms/step -
accuracy: 0.9725 - loss: 0.0916 - val_accuracy: 0.8940 - val_loss: 0.2662
Epoch 29/30
100/100              5s 46ms/step -
accuracy: 0.9745 - loss: 0.0928 - val_accuracy: 0.8940 - val_loss: 0.2674
Epoch 30/30
100/100              5s 46ms/step -
accuracy: 0.9770 - loss: 0.0871 - val_accuracy: 0.8880 - val_loss: 0.2689

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.


Training Q3_VGG16_noaug_5000samples | steps_per_epoch=250, epochs=30
Found 2000 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Epoch 1/30

I0000 00:00:1777518339.670734   23843 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_120800__.62

 99/250              5s 36ms/step -
accuracy: 0.5945 - loss: 0.7006

/home/solyvie/environments/miniforge3/envs/thesis-env/lib/python3.13/site-
packages/keras/src/trainers/epoch_iterator.py:116: UserWarning: Your input ran
out of data; interrupting training. Make sure that your dataset or generator can
generate at least `steps_per_epoch * epochs` batches. You may need to use the
`.repeat()` function when building your dataset.
  self._interrupted_warning()

250/250              10s 25ms/step -
accuracy: 0.6435 - loss: 0.6227 - val_accuracy: 0.8140 - val_loss: 0.4533
Epoch 2/30
250/250              5s 18ms/step -
```

```
accuracy: 0.8025 - loss: 0.4386 - val_accuracy: 0.8620 - val_loss: 0.3608
Epoch 3/30
250/250              5s 18ms/step -
accuracy: 0.8470 - loss: 0.3490 - val_accuracy: 0.8680 - val_loss: 0.3205
Epoch 4/30
250/250              5s 18ms/step -
accuracy: 0.8620 - loss: 0.3126 - val_accuracy: 0.8820 - val_loss: 0.2987
Epoch 5/30
250/250              5s 19ms/step -
accuracy: 0.8880 - loss: 0.2800 - val_accuracy: 0.8840 - val_loss: 0.2856
Epoch 6/30
250/250              5s 18ms/step -
accuracy: 0.8985 - loss: 0.2631 - val_accuracy: 0.8920 - val_loss: 0.2787
Epoch 7/30
250/250              5s 18ms/step -
accuracy: 0.9040 - loss: 0.2457 - val_accuracy: 0.8940 - val_loss: 0.2705
Epoch 8/30
250/250              5s 18ms/step -
accuracy: 0.9090 - loss: 0.2324 - val_accuracy: 0.8940 - val_loss: 0.2730
Epoch 9/30
250/250              5s 18ms/step -
accuracy: 0.9240 - loss: 0.2059 - val_accuracy: 0.8900 - val_loss: 0.2709
Epoch 10/30
250/250              5s 19ms/step -
accuracy: 0.9240 - loss: 0.2007 - val_accuracy: 0.9040 - val_loss: 0.2631
Epoch 11/30
250/250              5s 18ms/step -
accuracy: 0.9235 - loss: 0.1961 - val_accuracy: 0.8980 - val_loss: 0.2628
Epoch 12/30
250/250              5s 20ms/step -
accuracy: 0.9325 - loss: 0.1891 - val_accuracy: 0.8980 - val_loss: 0.2590
Epoch 13/30
250/250              5s 18ms/step -
accuracy: 0.9365 - loss: 0.1708 - val_accuracy: 0.9000 - val_loss: 0.2586
Epoch 14/30
250/250              5s 19ms/step -
accuracy: 0.9360 - loss: 0.1685 - val_accuracy: 0.8960 - val_loss: 0.2598
Epoch 15/30
250/250              5s 19ms/step -
accuracy: 0.9405 - loss: 0.1621 - val_accuracy: 0.9020 - val_loss: 0.2583
Epoch 16/30
250/250              5s 18ms/step -
accuracy: 0.9455 - loss: 0.1500 - val_accuracy: 0.8800 - val_loss: 0.2811
Epoch 17/30
250/250              5s 19ms/step -
accuracy: 0.9460 - loss: 0.1457 - val_accuracy: 0.9080 - val_loss: 0.2579
Epoch 18/30
250/250              5s 20ms/step -
```

```
accuracy: 0.9510 - loss: 0.1379 - val_accuracy: 0.9000 - val_loss: 0.2601
Epoch 19/30
250/250              5s 18ms/step -
accuracy: 0.9545 - loss: 0.1344 - val_accuracy: 0.9060 - val_loss: 0.2591
Epoch 20/30
250/250              5s 18ms/step -
accuracy: 0.9525 - loss: 0.1358 - val_accuracy: 0.9020 - val_loss: 0.2607
Epoch 21/30
250/250              5s 19ms/step -
accuracy: 0.9640 - loss: 0.1225 - val_accuracy: 0.8960 - val_loss: 0.2648
Epoch 22/30
250/250              5s 18ms/step -
accuracy: 0.9645 - loss: 0.1142 - val_accuracy: 0.9020 - val_loss: 0.2643
Epoch 23/30
250/250              5s 18ms/step -
accuracy: 0.9640 - loss: 0.1136 - val_accuracy: 0.9020 - val_loss: 0.2646
Epoch 24/30
250/250              5s 18ms/step -
accuracy: 0.9690 - loss: 0.1083 - val_accuracy: 0.9040 - val_loss: 0.2654
Epoch 25/30
250/250              5s 18ms/step -
accuracy: 0.9675 - loss: 0.1086 - val_accuracy: 0.8960 - val_loss: 0.2694
Epoch 26/30
250/250              5s 18ms/step -
accuracy: 0.9700 - loss: 0.1006 - val_accuracy: 0.9040 - val_loss: 0.2686
Epoch 27/30
250/250              5s 19ms/step -
accuracy: 0.9745 - loss: 0.0936 - val_accuracy: 0.9040 - val_loss: 0.2708
Epoch 28/30
250/250              5s 20ms/step -
accuracy: 0.9765 - loss: 0.0895 - val_accuracy: 0.9020 - val_loss: 0.2717
Epoch 29/30
250/250              5s 18ms/step -
accuracy: 0.9800 - loss: 0.0853 - val_accuracy: 0.8800 - val_loss: 0.2819
Epoch 30/30
250/250              5s 19ms/step -
accuracy: 0.9790 - loss: 0.0828 - val_accuracy: 0.8880 - val_loss: 0.2754
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
```

```
Training Q3_VGG16_noaug_10000samples | steps_per_epoch=500, epochs=30
Found 2000 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
```

```
Epoch 1/30

I0000 00:00:1777518490.835496   23845 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_139004__.62

500/500              11s 13ms/step -
accuracy: 0.6610 - loss: 0.6282 - val_accuracy: 0.8440 - val_loss: 0.4370
Epoch 2/30
500/500              5s 9ms/step -
accuracy: 0.8060 - loss: 0.4363 - val_accuracy: 0.8560 - val_loss: 0.3668
Epoch 3/30
500/500              5s 10ms/step -
accuracy: 0.8400 - loss: 0.3592 - val_accuracy: 0.8620 - val_loss: 0.3250
Epoch 4/30
500/500              5s 9ms/step -
accuracy: 0.8700 - loss: 0.3170 - val_accuracy: 0.8740 - val_loss: 0.3047
Epoch 5/30
500/500              5s 10ms/step -
accuracy: 0.8770 - loss: 0.2908 - val_accuracy: 0.8740 - val_loss: 0.2933
Epoch 6/30
500/500              5s 10ms/step -
accuracy: 0.8970 - loss: 0.2652 - val_accuracy: 0.9000 - val_loss: 0.2910
Epoch 7/30
500/500              5s 10ms/step -
accuracy: 0.9000 - loss: 0.2496 - val_accuracy: 0.8960 - val_loss: 0.2881
Epoch 8/30
500/500              5s 9ms/step -
accuracy: 0.9055 - loss: 0.2336 - val_accuracy: 0.8800 - val_loss: 0.2774
Epoch 9/30
500/500              5s 10ms/step -
accuracy: 0.9055 - loss: 0.2207 - val_accuracy: 0.8980 - val_loss: 0.2719
Epoch 10/30
500/500              5s 10ms/step -
accuracy: 0.9205 - loss: 0.2110 - val_accuracy: 0.8900 - val_loss: 0.2654
Epoch 11/30
500/500              5s 10ms/step -
accuracy: 0.9225 - loss: 0.2016 - val_accuracy: 0.8860 - val_loss: 0.2685
Epoch 12/30
500/500              5s 9ms/step -
accuracy: 0.9295 - loss: 0.1879 - val_accuracy: 0.8960 - val_loss: 0.2697
Epoch 13/30
500/500              5s 9ms/step -
accuracy: 0.9290 - loss: 0.1840 - val_accuracy: 0.8900 - val_loss: 0.2665
Epoch 14/30
500/500              5s 9ms/step -
accuracy: 0.9385 - loss: 0.1716 - val_accuracy: 0.8920 - val_loss: 0.2641
Epoch 15/30
500/500              5s 9ms/step -
accuracy: 0.9400 - loss: 0.1628 - val_accuracy: 0.9020 - val_loss: 0.2606
```

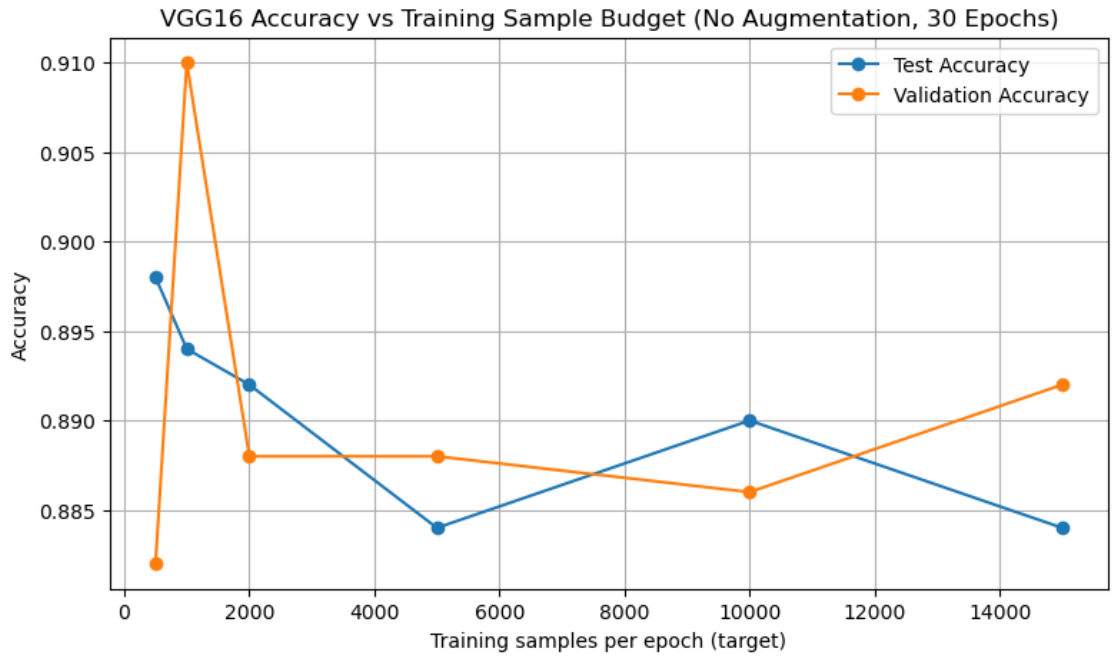
```
Epoch 16/30
500/500              5s 9ms/step -
accuracy: 0.9430 - loss: 0.1660 - val_accuracy: 0.9080 - val_loss: 0.2597
Epoch 17/30
500/500              5s 9ms/step -
accuracy: 0.9445 - loss: 0.1563 - val_accuracy: 0.9020 - val_loss: 0.2598
Epoch 18/30
500/500              5s 10ms/step -
accuracy: 0.9510 - loss: 0.1448 - val_accuracy: 0.8960 - val_loss: 0.2632
Epoch 19/30
500/500              5s 10ms/step -
accuracy: 0.9565 - loss: 0.1356 - val_accuracy: 0.9020 - val_loss: 0.2624
Epoch 20/30
500/500              5s 9ms/step -
accuracy: 0.9505 - loss: 0.1331 - val_accuracy: 0.9080 - val_loss: 0.2617
Epoch 21/30
500/500              5s 10ms/step -
accuracy: 0.9645 - loss: 0.1235 - val_accuracy: 0.8980 - val_loss: 0.2626
Epoch 22/30
500/500              5s 9ms/step -
accuracy: 0.9590 - loss: 0.1239 - val_accuracy: 0.9000 - val_loss: 0.2673
Epoch 23/30
500/500              5s 9ms/step -
accuracy: 0.9645 - loss: 0.1181 - val_accuracy: 0.9000 - val_loss: 0.2628
Epoch 24/30
500/500              5s 9ms/step -
accuracy: 0.9695 - loss: 0.1099 - val_accuracy: 0.9000 - val_loss: 0.2674
Epoch 25/30
500/500              5s 9ms/step -
accuracy: 0.9670 - loss: 0.1054 - val_accuracy: 0.9000 - val_loss: 0.2661
Epoch 26/30
500/500              5s 9ms/step -
accuracy: 0.9690 - loss: 0.0998 - val_accuracy: 0.8800 - val_loss: 0.2873
Epoch 27/30
500/500              5s 9ms/step -
accuracy: 0.9660 - loss: 0.1045 - val_accuracy: 0.8920 - val_loss: 0.2741
Epoch 28/30
500/500              5s 9ms/step -
accuracy: 0.9715 - loss: 0.0977 - val_accuracy: 0.8980 - val_loss: 0.2694
Epoch 29/30
500/500              4s 9ms/step -
accuracy: 0.9730 - loss: 0.0926 - val_accuracy: 0.8960 - val_loss: 0.2742
Epoch 30/30
500/500              5s 9ms/step -
accuracy: 0.9755 - loss: 0.0862 - val_accuracy: 0.8860 - val_loss: 0.2799
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
```

```
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.


Training Q3_VGG16_noaug_15000samples | steps_per_epoch=750, epochs=30
Found 2000 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Epoch 1/30

I0000 00:00:1777518642.892472   23845 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_157208__.62

750/750            11s 9ms/step -
accuracy: 0.6770 - loss: 0.5922 - val_accuracy: 0.8720 - val_loss: 0.4174
Epoch 2/30
750/750            5s 6ms/step -
accuracy: 0.8125 - loss: 0.4146 - val_accuracy: 0.8820 - val_loss: 0.3427
Epoch 3/30
750/750            5s 6ms/step -
accuracy: 0.8590 - loss: 0.3379 - val_accuracy: 0.8720 - val_loss: 0.3042
Epoch 4/30
750/750            5s 6ms/step -
accuracy: 0.8785 - loss: 0.3023 - val_accuracy: 0.8800 - val_loss: 0.2860
Epoch 5/30
750/750            5s 6ms/step -
accuracy: 0.8820 - loss: 0.2828 - val_accuracy: 0.8760 - val_loss: 0.2766
Epoch 6/30
750/750            5s 6ms/step -
accuracy: 0.8995 - loss: 0.2530 - val_accuracy: 0.8880 - val_loss: 0.2672
Epoch 7/30
750/750            5s 6ms/step -
accuracy: 0.9005 - loss: 0.2488 - val_accuracy: 0.8980 - val_loss: 0.2637
Epoch 8/30
750/750            5s 6ms/step -
accuracy: 0.9085 - loss: 0.2286 - val_accuracy: 0.8980 - val_loss: 0.2762
Epoch 9/30
750/750            5s 6ms/step -
accuracy: 0.9160 - loss: 0.2196 - val_accuracy: 0.9020 - val_loss: 0.2578
Epoch 10/30
750/750            5s 6ms/step -
accuracy: 0.9240 - loss: 0.1987 - val_accuracy: 0.8980 - val_loss: 0.2551
Epoch 11/30
750/750            5s 6ms/step -
accuracy: 0.9200 - loss: 0.1925 - val_accuracy: 0.9020 - val_loss: 0.2562
Epoch 12/30
750/750            5s 6ms/step -
accuracy: 0.9305 - loss: 0.1861 - val_accuracy: 0.8800 - val_loss: 0.2651
```

```
Epoch 13/30
750/750              5s 6ms/step -
accuracy: 0.9365 - loss: 0.1721 - val_accuracy: 0.8980 - val_loss: 0.2524
Epoch 14/30
750/750              5s 6ms/step -
accuracy: 0.9360 - loss: 0.1749 - val_accuracy: 0.8920 - val_loss: 0.2556
Epoch 15/30
750/750              5s 6ms/step -
accuracy: 0.9440 - loss: 0.1599 - val_accuracy: 0.8880 - val_loss: 0.2572
Epoch 16/30
750/750              5s 6ms/step -
accuracy: 0.9420 - loss: 0.1577 - val_accuracy: 0.9040 - val_loss: 0.2533
Epoch 17/30
750/750              5s 6ms/step -
accuracy: 0.9485 - loss: 0.1497 - val_accuracy: 0.8900 - val_loss: 0.2625
Epoch 18/30
750/750              5s 6ms/step -
accuracy: 0.9440 - loss: 0.1394 - val_accuracy: 0.8980 - val_loss: 0.2537
Epoch 19/30
750/750              5s 6ms/step -
accuracy: 0.9515 - loss: 0.1378 - val_accuracy: 0.8960 - val_loss: 0.2546
Epoch 20/30
750/750              5s 6ms/step -
accuracy: 0.9565 - loss: 0.1288 - val_accuracy: 0.8840 - val_loss: 0.2634
Epoch 21/30
750/750              5s 6ms/step -
accuracy: 0.9545 - loss: 0.1298 - val_accuracy: 0.8960 - val_loss: 0.2567
Epoch 22/30
750/750              5s 6ms/step -
accuracy: 0.9620 - loss: 0.1217 - val_accuracy: 0.9060 - val_loss: 0.2572
Epoch 23/30
750/750              5s 6ms/step -
accuracy: 0.9655 - loss: 0.1167 - val_accuracy: 0.8940 - val_loss: 0.2580
Epoch 24/30
750/750              5s 6ms/step -
accuracy: 0.9690 - loss: 0.1056 - val_accuracy: 0.8940 - val_loss: 0.2594
Epoch 25/30
750/750              5s 6ms/step -
accuracy: 0.9605 - loss: 0.1089 - val_accuracy: 0.8920 - val_loss: 0.2611
Epoch 26/30
750/750              5s 6ms/step -
accuracy: 0.9720 - loss: 0.0989 - val_accuracy: 0.9040 - val_loss: 0.2657
Epoch 27/30
750/750              5s 6ms/step -
accuracy: 0.9700 - loss: 0.0991 - val_accuracy: 0.8940 - val_loss: 0.2621
Epoch 28/30
750/750              5s 6ms/step -
accuracy: 0.9730 - loss: 0.0964 - val_accuracy: 0.8940 - val_loss: 0.2633
```

```
Epoch 29/30
750/750              5s 6ms/step -
accuracy: 0.9740 - loss: 0.0901 - val_accuracy: 0.8980 - val_loss: 0.2668
Epoch 30/30
750/750              5s 6ms/step -
accuracy: 0.9780 - loss: 0.0869 - val_accuracy: 0.8920 - val_loss: 0.2664

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

Q3 Results Table

   samples_per_epoch_target  steps_per_epoch  epochs_ran  final_train_acc  \
0                       500               25          30           0.9060
1                      1000               50          30           0.9550
2                      2000              100          30           0.9770
3                      5000              250          30           0.9790
4                     10000              500          30           0.9755
5                     15000              750          30           0.9780

   final_val_acc  final_test_acc  train_time_sec  \
0          0.882           0.898       64.612893
1          0.910           0.894       94.250703
2          0.888           0.892      146.686826
3          0.888           0.884      146.885737
4          0.886           0.890      149.757723
5          0.892           0.884      144.916862

                                      model_h5_path
0  /home/solyvie/workspace/cs-projects/25263CSDMa…
1  /home/solyvie/workspace/cs-projects/25263CSDMa…
2  /home/solyvie/workspace/cs-projects/25263CSDMa…
3  /home/solyvie/workspace/cs-projects/25263CSDMa…
4  /home/solyvie/workspace/cs-projects/25263CSDMa…
5  /home/solyvie/workspace/cs-projects/25263CSDMa…
```




```
Best Q3 run: 500 samples/epoch (test_acc=0.8980)
```

1.4 4) Implement Xception and Compare Architecture + Accuracy

Uses the same train/validation/test split configured at the start of this notebook.

[7]:
```
q4_xception = train_eval_save(
    model_name='Q4_Xception',
    backbone_name='Xception',
    train_aug_kwargs=best_aug,
    epochs=30,
    early_stopping=True,
)

# Build unified comparison table using best model per type.
q1_best_row = q1_df.sort_values('final_test_acc', ascending=False).iloc[0]
q2_best_row = q2_df.sort_values('final_test_acc', ascending=False).iloc[0]

# Architecture metadata
arch_meta = {
    'VGG16': {
        'num_layers': len(VGG16(weights='imagenet', include_top=False,␣
 ↪input_shape=(150, 150, 3)).layers),
        'num_params': VGG16(weights='imagenet', include_top=False,␣
 ↪input_shape=(150, 150, 3)).count_params(),
    },
```

```
    'VGG19': {
        'num_layers': len(VGG19(weights='imagenet', include_top=False,␣
 ↪input_shape=(150, 150, 3)).layers),
        'num_params': VGG19(weights='imagenet', include_top=False,␣
 ↪input_shape=(150, 150, 3)).count_params(),
    },
    'Xception': {
        'num_layers': len(Xception(weights='imagenet', include_top=False,␣
 ↪input_shape=(150, 150, 3)).layers),
        'num_params': Xception(weights='imagenet', include_top=False,␣
 ↪input_shape=(150, 150, 3)).count_params(),
    },
}

q4_compare = pd.DataFrame([
    {
        'model_type': 'Best VGG16/VGG19 from Q2',
        'backbone': q2_best_row['backbone'],
        'num_layers': arch_meta[q2_best_row['backbone']]['num_layers'],
        'num_params': arch_meta[q2_best_row['backbone']]['num_params'],
        'final_val_acc': q2_best_row['final_val_acc'],
        'final_test_acc': q2_best_row['final_test_acc'],
        'model_h5_path': q2_best_row['model_h5_path'],
    },
    {
        'model_type': 'Best augmentation VGG16 from Q1',
        'backbone': 'VGG16',
        'num_layers': arch_meta['VGG16']['num_layers'],
        'num_params': arch_meta['VGG16']['num_params'],
        'final_val_acc': q1_best_row['final_val_acc'],
        'final_test_acc': q1_best_row['final_test_acc'],
        'model_h5_path': q1_best_row['model_h5_path'],
    },
    {
        'model_type': 'Xception (Q4)',
        'backbone': 'Xception',
        'num_layers': arch_meta['Xception']['num_layers'],
        'num_params': arch_meta['Xception']['num_params'],
        'final_val_acc': q4_xception['final_val_acc'],
        'final_test_acc': q4_xception['final_test_acc'],
        'model_h5_path': q4_xception['model_h5_path'],
    },
])

print('Q4 Comparison Table')
display(q4_compare.sort_values('final_test_acc', ascending=False).
 ↪reset_index(drop=True))
```

```
q4_best = q4_compare.sort_values('final_test_acc', ascending=False).iloc[0]
print(
    f"Best overall in Q4 comparison: {q4_best['backbone']} "
    f"(test_acc={q4_best['final_test_acc']:.4f})"
)
```

```
Found 2000 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Found 500 images belonging to 2 classes.
Epoch 1/30
```

```
I0000 00:00:1777518795.806049   23842 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_181170__.123
E0000 00:00:1777518798.086268   23842 cuda_timer.cc:87] Delay kernel timed out:
measured time has sub-optimal accuracy. There may be a missing warmup execution,
please investigate in Nsight Systems.
E0000 00:00:1777518800.221768   23842 cuda_timer.cc:87] Delay kernel timed out:
measured time has sub-optimal accuracy. There may be a missing warmup execution,
please investigate in Nsight Systems.
E0000 00:00:1777518804.922548   23842 cuda_timer.cc:87] Delay kernel timed out:
measured time has sub-optimal accuracy. There may be a missing warmup execution,
please investigate in Nsight Systems.
E0000 00:00:1777518806.268170   23842 cuda_timer.cc:87] Delay kernel timed out:
measured time has sub-optimal accuracy. There may be a missing warmup execution,
please investigate in Nsight Systems.
```

```
100/100              44s 176ms/step -
accuracy: 0.8025 - loss: 0.4424 - val_accuracy: 0.9500 - val_loss: 0.2286
Epoch 2/30
100/100              13s 127ms/step -
accuracy: 0.9115 - loss: 0.2431 - val_accuracy: 0.9600 - val_loss: 0.1527
Epoch 3/30
100/100              12s 115ms/step -
accuracy: 0.9260 - loss: 0.1948 - val_accuracy: 0.9600 - val_loss: 0.1249
Epoch 4/30
100/100              13s 134ms/step -
accuracy: 0.9310 - loss: 0.1666 - val_accuracy: 0.9660 - val_loss: 0.1122
Epoch 5/30
100/100              12s 115ms/step -
accuracy: 0.9450 - loss: 0.1509 - val_accuracy: 0.9660 - val_loss: 0.1059
Epoch 6/30
100/100              13s 125ms/step -
accuracy: 0.9375 - loss: 0.1489 - val_accuracy: 0.9640 - val_loss: 0.1015
Epoch 7/30
100/100              20s 122ms/step -
accuracy: 0.9360 - loss: 0.1374 - val_accuracy: 0.9620 - val_loss: 0.0971
Epoch 8/30
100/100              13s 128ms/step -
```

```
accuracy: 0.9425 - loss: 0.1358 - val_accuracy: 0.9660 - val_loss: 0.0932
Epoch 9/30
100/100             13s 124ms/step -
accuracy: 0.9460 - loss: 0.1300 - val_accuracy: 0.9660 - val_loss: 0.0902
Epoch 10/30
100/100             13s 125ms/step -
accuracy: 0.9420 - loss: 0.1220 - val_accuracy: 0.9660 - val_loss: 0.0898
Epoch 11/30
100/100             14s 138ms/step -
accuracy: 0.9540 - loss: 0.1093 - val_accuracy: 0.9620 - val_loss: 0.0900
Epoch 12/30
100/100             13s 132ms/step -
accuracy: 0.9550 - loss: 0.1180 - val_accuracy: 0.9620 - val_loss: 0.0874
Epoch 13/30
100/100             12s 122ms/step -
accuracy: 0.9495 - loss: 0.1117 - val_accuracy: 0.9620 - val_loss: 0.0876
Epoch 14/30
100/100             14s 137ms/step -
accuracy: 0.9555 - loss: 0.1069 - val_accuracy: 0.9620 - val_loss: 0.0861
Epoch 15/30
100/100             14s 136ms/step -
accuracy: 0.9545 - loss: 0.1072 - val_accuracy: 0.9640 - val_loss: 0.0853
Epoch 16/30
100/100             13s 130ms/step -
accuracy: 0.9590 - loss: 0.1046 - val_accuracy: 0.9640 - val_loss: 0.0864
Epoch 17/30
100/100             13s 128ms/step -
accuracy: 0.9580 - loss: 0.1046 - val_accuracy: 0.9620 - val_loss: 0.0864
Epoch 18/30
100/100             13s 125ms/step -
accuracy: 0.9505 - loss: 0.1130 - val_accuracy: 0.9660 - val_loss: 0.0838
Epoch 19/30
100/100             13s 128ms/step -
accuracy: 0.9595 - loss: 0.1037 - val_accuracy: 0.9620 - val_loss: 0.0845
Epoch 20/30
100/100             12s 124ms/step -
accuracy: 0.9650 - loss: 0.0949 - val_accuracy: 0.9640 - val_loss: 0.0854
Epoch 21/30
100/100             14s 136ms/step -
accuracy: 0.9560 - loss: 0.1027 - val_accuracy: 0.9660 - val_loss: 0.0839
Epoch 22/30
100/100             13s 132ms/step -
accuracy: 0.9625 - loss: 0.0899 - val_accuracy: 0.9640 - val_loss: 0.0829
Epoch 23/30
100/100             13s 132ms/step -
accuracy: 0.9600 - loss: 0.0946 - val_accuracy: 0.9640 - val_loss: 0.0834
Epoch 24/30
100/100             14s 136ms/step -
```

```
accuracy: 0.9580 - loss: 0.0995 - val_accuracy: 0.9620 - val_loss: 0.0825
Epoch 25/30
100/100             13s 128ms/step -
accuracy: 0.9545 - loss: 0.0980 - val_accuracy: 0.9640 - val_loss: 0.0825
Epoch 26/30
100/100             13s 132ms/step -
accuracy: 0.9615 - loss: 0.0927 - val_accuracy: 0.9620 - val_loss: 0.0814
Epoch 27/30
100/100             13s 130ms/step -
accuracy: 0.9555 - loss: 0.0964 - val_accuracy: 0.9620 - val_loss: 0.0829
Epoch 28/30
100/100             13s 132ms/step -
accuracy: 0.9680 - loss: 0.0783 - val_accuracy: 0.9640 - val_loss: 0.0863
Epoch 29/30
100/100             13s 134ms/step -
accuracy: 0.9650 - loss: 0.0887 - val_accuracy: 0.9640 - val_loss: 0.0833
Epoch 30/30
100/100             12s 121ms/step -
accuracy: 0.9610 - loss: 0.0950 - val_accuracy: 0.9640 - val_loss: 0.0840
Restoring model weights from the end of the best epoch: 26.

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

Q4 Comparison Table

                       model_type  backbone  num_layers  num_params  \
0                    Xception (Q4)  Xception         132    20861480
1         Best VGG16/VGG19 from Q2     VGG19          22    20024384
2  Best augmentation VGG16 from Q1     VGG16          19    14714688

   final_val_acc  final_test_acc  \
0          0.962           0.964
1          0.872           0.910
2          0.898           0.900

                                      model_h5_path
0  /home/solyvie/workspace/cs-projects/25263CSDMa…
1  /home/solyvie/workspace/cs-projects/25263CSDMa…
2  /home/solyvie/workspace/cs-projects/25263CSDMa…

Best overall in Q4 comparison: Xception (test_acc=0.9640)
```

1.5 Example Classification on Unseen/Test Data

[8]:
```
# Pick best available model by test accuracy from Q2/Q4 and run one example␣
↪prediction.
candidate_runs = [
    ('Q2_VGG16', q2_vgg16),
    ('Q2_VGG19', q2_vgg19),
    ('Q4_Xception', q4_xception),
]
best_name, best_run = sorted(candidate_runs, key=lambda x:␣
↪x[1]['final_test_acc'], reverse=True)[0]
best_model = best_run['model_obj']

# Get one random image from test split.
all_test_images = []
for cls in ['cats', 'dogs']:
    cls_dir = test_dir / cls
    all_test_images.extend([(p, cls) for p in cls_dir.iterdir() if p.is_file()])

img_path, true_label = random.choice(all_test_images)
img = keras_image.load_img(str(img_path), target_size=IMG_SIZE)
img_arr = keras_image.img_to_array(img) / 255.0
img_arr = np.expand_dims(img_arr, axis=0)

pred_score = float(best_model.predict(img_arr, verbose=0)[0][0])
pred_label = 'dog' if pred_score >= 0.5 else 'cat'
confidence = pred_score if pred_label == 'dog' else (1 - pred_score)

plt.figure(figsize=(4, 4))
plt.imshow(img)
plt.title(
    f'Model: {best_name}\nTrue: {true_label} | Pred: {pred_label} | Conf:␣
↪{confidence:.3f}'
)
plt.axis('off')
plt.show()

print('Example image path:', img_path)
print('Best model used:', best_name)
print('Pred score (dog probability):', pred_score)
```

```
E0000 00:00:1777519230.840526   23842 cuda_timer.cc:87] Delay kernel timed out:
measured time has sub-optimal accuracy. There may be a missing warmup execution,
please investigate in Nsight Systems.
E0000 00:00:1777519235.062723   23842 cuda_timer.cc:87] Delay kernel timed out:
measured time has sub-optimal accuracy. There may be a missing warmup execution,
please investigate in Nsight Systems.
E0000 00:00:1777519238.765387   23842 cuda_timer.cc:87] Delay kernel timed out:
```

```
measured time has sub-optimal accuracy. There may be a missing warmup execution,
please investigate in Nsight Systems.
E0000 00:00:1777519242.503528   23842 cuda_timer.cc:87] Delay kernel timed out:
measured time has sub-optimal accuracy. There may be a missing warmup execution,
please investigate in Nsight Systems.
```

```
Example image path: /home/solyvie/workspace/cs-
projects/25263CSDMachineLearning/[7] Group 8 - Module 6 CNN Part 3 - Using Pre-
Trained Networks/data/working_split_for_lab3/test/dogs/8191.jpg
Best model used: Q4_Xception
Pred score (dog probability): 0.9998792409896851
```

1.6 Final Insights (Auto-generated from your run)

[9]:
```
# Summaries and exports for reporting.
summary = {
    'q1': q1_df.drop(columns=['history', 'model_obj'], errors='ignore').
 ↪to_dict(orient='records'),
    'q2': q2_df.drop(columns=['history', 'model_obj'], errors='ignore').
 ↪to_dict(orient='records'),
    'q3': q3_df.to_dict(orient='records'),
    'q4': {
        'xception': {k: v for k, v in q4_xception.items() if k not in␣
 ↪['history', 'model_obj']},
        'comparison_table': q4_compare.to_dict(orient='records'),
```

```
    },
    'dataset_split_note': split_note,
}

summary_json = OUTPUT_DIR / 'Group8_ANN_Lab3_summary.json'
with open(summary_json, 'w', encoding='utf-8') as f:
    import json
    json.dump(summary, f, indent=2)

q1_csv = OUTPUT_DIR / 'Group8_ANN_Lab3_q1_table.csv'
q2_csv = OUTPUT_DIR / 'Group8_ANN_Lab3_q2_table.csv'
q3_csv = OUTPUT_DIR / 'Group8_ANN_Lab3_q3_table.csv'
q4_csv = OUTPUT_DIR / 'Group8_ANN_Lab3_q4_table.csv'

q1_df.drop(columns=['history', 'model_obj'], errors='ignore').to_csv(q1_csv,␣
 ↪index=False)
q2_df.drop(columns=['history', 'model_obj'], errors='ignore').to_csv(q2_csv,␣
 ↪index=False)
q3_df.to_csv(q3_csv, index=False)
q4_compare.to_csv(q4_csv, index=False)

print('Saved summary JSON:', summary_json)
print('Saved tables:')
print(' -', q1_csv)
print(' -', q2_csv)
print(' -', q3_csv)
print(' -', q4_csv)

# Textual insights driven by metrics from this exact run.
q1_best = q1_df.sort_values('final_test_acc', ascending=False).iloc[0]
q2_best = q2_df.sort_values('final_test_acc', ascending=False).iloc[0]
q3_best = q3_df.sort_values('final_test_acc', ascending=False).iloc[0]
q4_best = q4_compare.sort_values('final_test_acc', ascending=False).iloc[0]

print('\n=== INSIGHTS ===')
print(f"Q1 Best Augmentation: {q1_best['model_name']}␣
 ↪(test_acc={q1_best['final_test_acc']:.4f})")
print(f"Q2 Best Backbone:     {q2_best['backbone']}␣
 ↪(test_acc={q2_best['final_test_acc']:.4f})")
print(
    f"Q3 Best Sample Budget: {int(q3_best['samples_per_epoch_target'])} samples/
 ↪epoch "
    f"(test_acc={q3_best['final_test_acc']:.4f})"
)
print(f"Q4 Best Overall:      {q4_best['backbone']}␣
 ↪(test_acc={q4_best['final_test_acc']:.4f})")
```

```python
# Diminishing-return diagnostic for Q3
q3_sorted = q3_df.sort_values('samples_per_epoch_target').reset_index(drop=True)
q3_sorted['delta_test_acc'] = q3_sorted['final_test_acc'].diff()
print('\nQ3 marginal gains (test accuracy delta):')
display(q3_sorted[['samples_per_epoch_target', 'final_test_acc',␣
↪'delta_test_acc']])
```

```
Saved summary JSON: /home/solyvie/workspace/cs-
projects/25263CSDMachineLearning/[7] Group 8 - Module 6 CNN Part 3 - Using Pre-
Trained Networks/models/Group8_ANN_Lab3_summary.json
Saved tables:
 - /home/solyvie/workspace/cs-projects/25263CSDMachineLearning/[7] Group 8 -
Module 6 CNN Part 3 - Using Pre-Trained
Networks/models/Group8_ANN_Lab3_q1_table.csv
 - /home/solyvie/workspace/cs-projects/25263CSDMachineLearning/[7] Group 8 -
Module 6 CNN Part 3 - Using Pre-Trained
Networks/models/Group8_ANN_Lab3_q2_table.csv
 - /home/solyvie/workspace/cs-projects/25263CSDMachineLearning/[7] Group 8 -
Module 6 CNN Part 3 - Using Pre-Trained
Networks/models/Group8_ANN_Lab3_q3_table.csv
 - /home/solyvie/workspace/cs-projects/25263CSDMachineLearning/[7] Group 8 -
Module 6 CNN Part 3 - Using Pre-Trained
Networks/models/Group8_ANN_Lab3_q4_table.csv

=== INSIGHTS ===
Q1 Best Augmentation: Q1_VGG16_default_aug (test_acc=0.9000)
Q2 Best Backbone:     VGG19 (test_acc=0.9100)
Q3 Best Sample Budget: 500 samples/epoch (test_acc=0.8980)
Q4 Best Overall:      Xception (test_acc=0.9640)

Q3 marginal gains (test accuracy delta):
```

	samples_per_epoch_target	final_test_acc	delta_test_acc
0	500	0.898	NaN
1	1000	0.894	-0.004
2	2000	0.892	-0.002
3	5000	0.884	-0.008
4	10000	0.890	0.006
5	15000	0.884	-0.006

1.7 Export to PDF

After running all cells, export the notebook as PDF:

```
jupyter nbconvert --to pdf Group8_ANN_Lab3.ipynb
```